Shell 编程

一、概述

1、Shell 脚本

2、Shell 环境

①、bash 和 sh 的关系

②、Centos 默认的解析器是 bash

3、概述

二、Shell 脚本入门

1、脚本格式

2、第一个 Shell 脚本：helloworld

①、需求

②、案例实操

③、脚本的常用执行方式

Ⅰ、采用 bash 或 sh + 脚本的相对路径或绝对路径（不用赋予脚本+x 权限）

(1)、sh+脚本的相对路径

(2)、sh+脚本的绝对路径

(3)、bash+脚本的相对路径

(4)、bash+脚本的绝对路径

Ⅱ、采用输入脚本的绝对路径或相对路径执行脚本（必须具有可执行权限+x）

(1)、相对路径

(2)、绝对路径

Ⅲ、在脚本的路径前加上 "." 或者 source

(1)、有以下脚本

(2)、绝对路径

(2)、原因

三、Shell 变量

1、系统预定义变量

①、查看系统变量的值

②、显示当前 Shell 中所有变量：set

2、自定义变量
①、基本语法
Ⅰ、定义变量
Ⅱ、调用变量
Ⅲ、将变量提升为全局变量
Ⅳ、撤销变量
Ⅴ、声明静态变量（常量）
②、变量定义规则
3、特殊变量
①、$n
Ⅰ、语法
Ⅱ、案例
②、$#
Ⅰ、语法
Ⅱ、案例
③、$*
Ⅰ、语法
Ⅱ、案例
④、$@
Ⅰ、语法
Ⅱ、案例
⑤、$?
Ⅰ、语法
Ⅱ、案例
四、运算符
1、语法
2、案例
五、条件判断
1、基本语法
①、test condition
②、[condition]
2、常用判断条件

①、两个整数之间比较
②、按照文件权限进行判断
③、按照文件类型进行判断
④、与 && 或 || 操作
Ⅰ、逻辑与：&&
Ⅱ、逻辑或：||
Ⅲ、三元运算符
六、流程控制
1、if 判断
①、语法
Ⅰ、单分支
Ⅱ、多分支
②、注意事项
③、案例
2、case 语句
①、语法
②、注意事项
③、案例
3、while 循环
①、语法
②、注意事项
③、案例
4、for 循环
①、语法
②、注意事项：$* 和 $@ 的区别
③、案例
七、read 读取控制台输入
1、语法
①、参数
②、变量名
2、案例
八、函数

1、系统函数
①、 basename
Ⅰ、语法
Ⅱ、参数
Ⅲ、案例
②、 dirname
Ⅰ、语法
Ⅱ、参数
Ⅲ、案例
2、自定义函数
①、 语法
②、 经验技巧
③、案例
九、文本处理工具
1、cut
①、语法
②、参数
③、说明
④、案例
2、awk
十、综合应用案例
1、归档（备份）文件
2、发送消息

一、概述

Shell 是一个用 C 语言编写的程序，它是用户使用 Linux 的桥梁。Shell 既是一种命令语言，又是一种程序设计语言

Shell 是指一种应用程序，这个应用程序提供了一个界面，用户通过这个界面访问操作系统内核的服务

1、Shell 脚本

Shell 脚本（shell script），是一种为 shell 编写的脚本程序

业界所说的 shell 通常都是指 shell 脚本，但要知道，shell 和 shell script 是两个不同的概念

2、Shell 环境

Shell 编程跟 JavaScript、php 编程一样，只要有一个能编写代码的文本编辑器和一个能解释执行的脚本解释器就可以了。

Linux 的 Shell 种类众多，常见的有：

- Bourne Shell（/usr/bin/sh或/bin/sh）
- Bourne Again Shell（/bin/bash）
- C Shell（/usr/bin/csh）
- K Shell（/usr/bin/ksh）
- Shell for Root（/sbin/sh）
- ……

Shell | 复制代码

```shell
[trial@localhost bin]$ cat /etc/shells
/bin/sh
/bin/bash
/sbin/nologin
/bin/tcsh
/bin/csh
/bin/ksh
```

本教程关注的是 Bash，也就是 Bourne Again Shell，由于易用和免费，Bash 在日常工作中被广泛使用。同时，Bash 也是大多数Linux 系统默认的 Shell。

在一般情况下，人们并不区分 Bourne Shell 和 Bourne Again Shell，所以，像 **#!/bin/sh**，它同样也可以改为 **#!/bin/bash**。

`#!` 告诉系统其后路径所指定的程序即是解释此脚本文件的 Shell 程序

①、bash 和 sh 的关系

```shell
[trial@localhost shell]$ cd /bin
[trial@localhost bin]$ ll | grep bash
-rwxr-xr-x 1 root root  801528 Jul 10  2013 bash
lrwxrwxrwx 1 root root       4 Aug  8  2013 sh -> bash
```

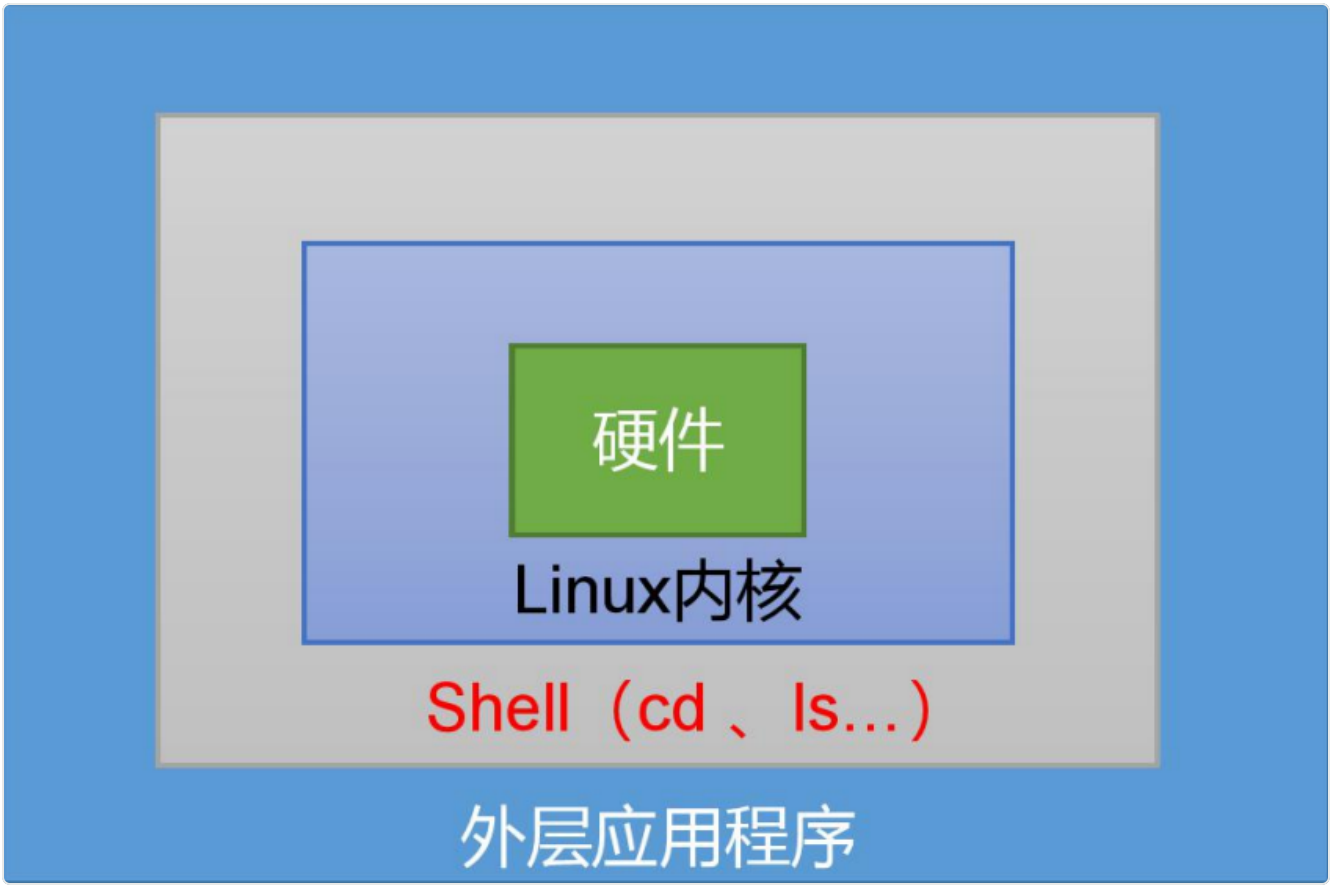
②、Centos 默认的解析器是 bash

```shell
[trial@localhost shell]$ cd /bin
[trial@localhost bin]$ echo $SHELL
/bin/bash
```

3、概述

Shell是一个命令行解释器，它接收应用程序/用户命令，然后调用操作系统内核。

Shell还是一个功能相当强大的编程语言，易编写、易调试、灵活性强。



二、Shell 脚本入门

1、脚本格式

脚本以 `#!/bin/bash` 开头（指定解析器）

2、第一个 Shell 脚本：helloworld

①、需求

创建一个 Shell 脚本，输出：helloworld

②、案例实操

1. 使用 `vim` 新建文件
2. 输入 shell 内容

Shell | 复制代码

```shell
#!/bin/bash
echo "Hello World !"
```

3. 保存并退出，给文件命名

Shell | 复制代码

```shell
:wq 00test.sh
```

4. 更改脚本权限

```shell
[trial@localhost shell]$ ll
total 4
-rw-rw-r-- 1 trial trial 33 Aug 10 16:50 00test.sh

[trial@localhost shell]$ chmod 755 00test.sh

[trial@localhost shell]$ ll
total 4
-rwxr-xr-x 1 trial trial 33 Aug 10 16:50 test01.sh
```

5. 执行脚本

```shell
[trial@localhost shell]$ ./00test.sh
Hello World !
```

③、脚本的常用执行方式

Ⅰ、采用 bash 或 sh + 脚本的相对路径或绝对路径（不用赋予脚本+x 权限）

(1)、sh+脚本的相对路径

```shell
[trial@localhost shell]$ sh ./test01.sh
Hello World !
```

(2)、sh+脚本的绝对路径

```shell
[trial@localhost shell]$ sh /home/trial/work/cuichangjian/shell/test01.sh
Hello World !
```

(3)、bash+脚本的相对路径

```shell
[trial@localhost shell]$ bash ./test01.sh
Hello World !
```

(4)、bash+脚本的绝对路径

```shell
[trial@localhost shell]$ bash /home/trial/work/cuichangjian/shell/test01.sh
Hello World !
```

II 、采用输入脚本的绝对路径或相对路径执行脚本（必须具有可执行权限+x）

(1)、相对路径

```shell
[trial@localhost shell]$ ./test01.sh
Hello World !
```

(2)、绝对路径

```shell
[trial@localhost shell]$ pwd
/home/trial/work/cuichangjian/shell
[trial@localhost shell]$ /home/trial/work/cuichangjian/shell/test01.sh
Hello World !
```

III、在脚本的路径前加上 "." 或者 source

(1)、有以下脚本

```shell
[atguigu@hadoop101 shells]$ cat test.sh
#!/bin/bash
A=5
echo $A
```

(2)、绝对路径

```shell
[atguigu@hadoop101 shells]$ bash test.sh
[atguigu@hadoop101 shells]$ echo $A

[atguigu@hadoop101 shells]$ sh test.sh
[atguigu@hadoop101 shells]$ echo $A

[atguigu@hadoop101 shells]$ ./test.sh
[atguigu@hadoop101 shells]$ echo $A

[atguigu@hadoop101 shells]$ . test.sh
[atguigu@hadoop101 shells]$ echo $A
 5
```

(2)、原因

1. 前三种方式都是在当前 shell 中打开一个子 shell 来执行脚本内容，当脚本内容结束，则子 shell 关闭，回到父 shell 中
2. 第四种，也就是使用在脚本路径前加 "." 或者 source 的方式，可以使脚本内容在当前 shell 里执行，而无需打开子 shell。这也是为什么我们每次要修改完 /etc/profile 文件以后，需要 source 一下的原因
3. 开子 shell 与不开子 shell 的区别就在于，环境变量的继承关系，如在子 shell 中设置的当前变量，父 shell 是不可见的

三、Shell 变量

1、系统预定义变量

1. `$HOME`： 当前主目录
2. `$PWD`： 当前工作目录
3. `$SHELL`： 使用的 shell 解析器
4. `$USER`： 当前用户
5. 等

①、查看系统变量的值

```shell
[trial@localhost cuichangjian]$ echo $HOME
/home/trial
[trial@localhost cuichangjian]$ echo $PWD
/home/trial/work/cuichangjian
[trial@localhost cuichangjian]$ echo $SHELL
/bin/bash
[trial@localhost cuichangjian]$ echo $USER
trial
```

②、显示当前 Shell 中所有变量：`set`

Shell | 复制代码

```shell
[trial@localhost shell]$ set
BASH=/bin/bash
BASH_ARGC=()
BASH_ARGV=()
BASH_LINENO=()
BASH_SOURCE=()
BASH_VERSINFO=([0]="3" [1]="2" [2]="25" [3]="1" [4]="release" [5]="x86_64-redhat-linux-gnu")
BASH_VERSION='3.2.25(1)-release'
CASSPATH=.:/usr/java/jdk1.7.0_40/jre/lib/tools.jar
CLASSPATH=.:/usr/java/jdk1.7.0_40/lib/dt.jar:/usr/java/jdk1.7.0_40/lib/tools.jar
COLORS=/etc/DIR_COLORS
COLUMNS=85
DIRSTACK=()
EUID=502
GROUPS=()
G_BROKEN_FILENAMES=1
HISTFILE=/home/trial/.bash_history
HISTFILESIZE=1000
HISTSIZE=1000
HOME=/home/trial
HOSTNAME=localhost.localdomain
HOSTTYPE=x86_64
IFS=$' \t\n'
INPUTRC=/etc/inputrc
JAVA_HOME=/usr/java/jdk1.7.0_40
LANG=en_US.UTF-8
LESSOPEN='|/usr/bin/lesspipe.sh %s'
LINES=31
LOGNAME=trial
LS_COLORS=
MACHTYPE=x86_64-redhat-linux-gnu
MAIL=/var/spool/mail/trial
MAILCHECK=60
OLDPWD=/home/trial/work/cuichangjian
OPTERR=1
OPTIND=1
OSTYPE=linux-gnu
PATH=/home/SMART_TRIAL:/home/SMART:/usr/local/bin:/usr/kerberos/sbin:/usr/kerberos/bin:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:/root/bin:/home/trial/bin:/usr/local/samba/bin:/usr/local/samba/sbin
PIPESTATUS=([0]="0")
PPID=25100

```

```
PROMPT_COMMAND='printf "\033]0;%s@%s:%s\007" "${USER}" "${HOSTNAME%%.*}" "${PWD/#$HOME/~}"'
PS1='[\u@\h \W]\$ '
PS2='> '
PS4='+ '
PWD=/home/trial/work/cuichangjian/shell
SHELL=/bin/bash
SHELLOPTS=braceexpand:emacs:hashall:histexpand:history:interactive-comments:monitor
SHLVL=1
SSH_ASKPASS=/usr/libexec/openssh/gnome-ssh-askpass
SSH_CLIENT='172.20.2.88 5063 22'
SSH_CONNECTION='172.20.2.88 5063 172.20.2.44 22'
SSH_TTY=/dev/pts/5
TERM=xterm-256color
TOMCAT_HOME=/usr/local/tomcat
UID=502
USER=trial
_=/home/trial
consoletype=pty
tmpid=502
```

2、自定义变量

①、基本语法

Ⅰ、定义变量

`变量名=变量值`

注意，=号前后不能有空格

Shell | 复制代码

```shell
[trial@localhost cuichangjian]$ yuehai=16
[trial@localhost cuichangjian]$ echo $yuehai
16
```

---

1. 变量是和当前 shell 绑定的，在新 shell 中不存在之前 shell 自定义的变量（和之前使用 `.` 执行脚本一样）

```
[trial@localhost cuichangjian]$ yuehai=16
[trial@localhost cuichangjian]$ echo $yuehai
16
[trial@localhost cuichangjian]$
```

```
1.trial - 学习    2.trial - 学习:2
ssh ▸ trial - 学习
Last login: Fri Aug 12 09:37:31 2022 from 172.20.2.84
ようこそ、青島のLINUX練習サーバです。
[trial@localhost ~]$ echo $yuehai

[trial@localhost ~]$
```

2. 更改变量的值： `变量名=值`

```shell
[trial@localhost cuichangjian]$ echo $yuehai

[trial@localhost cuichangjian]$ yuehai=16
[trial@localhost cuichangjian]$ echo $yuehai
16
[trial@localhost cuichangjian]$ yuehai=18
[trial@localhost cuichangjian]$ echo $yuehai
18
```

II 、调用变量

`$变量名`

未定义的变量也可以调用，不过会为空

```shell
[trial@localhost cuichangjian]$ yuehai=16
[trial@localhost cuichangjian]$ echo $yuehai
16
[trial@localhost cuichangjian]$ echo $q

[trial@localhost cuichangjian]$
```

III、将变量提升为全局变量

`export 变量名`

1. 提升为全局变量后，在父 bash 和 子 bash 中都可以查看

2. 但若是在子 bash 中更改，并不会反应到父 bash 中；即子 bash 中的变量是更改后的，父 bash 中还是原来的
3. 之前执行脚本的方式：
   a. 相对绝对路径：`./脚本名`：在子 bash 中运行，即要提升为全局变量才可以调用父脚本的变量
   b. 在脚本的路径前加上 "." 或者 source：在父脚本中直接运行，不会运行子 bash，也就不用提升为全局变量

```
[trial@localhost cuichangjian]$ echo $yuehai                打印变量值，未定义，为空

[trial@localhost cuichangjian]$ yuehai=16                   定义变量
[trial@localhost cuichangjian]$ echo $yuehai                打印变量值
16
[trial@localhost cuichangjian]$ ps -f                       显示当前进程的状态
UID        PID  PPID  C STIME TTY          TIME CMD
trial    13542 13536  0 10:16 pts/6    00:00:00 -bash      → 现在在 -bash 下，端口号为 13542
trial    20778 13542  0 10:18 pts/6    00:00:00 ps -f
[trial@localhost cuichangjian]$ bash                        进入子 bash
[trial@localhost cuichangjian]$ ps -f                       显示当前进程的状态
UID        PID  PPID  C STIME TTY          TIME CMD
trial    13542 13536  0 10:16 pts/6    00:00:00 -bash
trial    21225 13542  0 10:18 pts/6    00:00:00 bash       → 现在在 -bash 的 子 bash 下，父端口号为 13542
trial    21476 21225  0 10:18 pts/6    00:00:00 ps -f
[trial@localhost cuichangjian]$ echo $yuehai                子 bash 不能调用父 bash 的变量

[trial@localhost cuichangjian]$ exit                        退出子 bash
exit
[trial@localhost cuichangjian]$ ps -f                       现在在父 bash 中
UID        PID  PPID  C STIME TTY          TIME CMD
trial    13542 13536  0 10:16 pts/6    00:00:00 -bash
trial    23629 13542  0 10:18 pts/6    00:00:00 ps -f
[trial@localhost cuichangjian]$ echo $yuehai                可以调用当前定义的变量
16
[trial@localhost cuichangjian]$ export yuehai               将变量提升为全局变量
[trial@localhost cuichangjian]$ echo $yuehai
16
[trial@localhost cuichangjian]$ bash                        进入子 bash
[trial@localhost cuichangjian]$ ps -f                       现在在子 bash 中
UID        PID  PPID  C STIME TTY          TIME CMD
trial    13542 13536  0 10:16 pts/6    00:00:00 -bash
trial    26673 13542  0 10:19 pts/6    00:00:00 bash
trial    26925 26673  0 10:19 pts/6    00:00:00 ps -f
[trial@localhost cuichangjian]$ echo $yuehai                现在在子 bash 中就可以调用
16                                                          父 bash 中定义的变量（全局变量）
[trial@localhost cuichangjian]$
```

IV、撤销变量

`unset 变量名`

```shell
[trial@localhost cuichangjian]$ echo $yuehai

[trial@localhost cuichangjian]$ yuehai=16
[trial@localhost cuichangjian]$ echo $yuehai
16
[trial@localhost cuichangjian]$ unset yuehai
[trial@localhost cuichangjian]$ echo $yuehai

[trial@localhost cuichangjian]$
```

V、声明静态变量（常量）

`readonly 变量名`

注意：静态不能更改值、不能 unset

```shell
[trial@localhost cuichangjian]$ readonly yan=14
[trial@localhost cuichangjian]$ echo $yan
14
[trial@localhost cuichangjian]$ yan=12
bash: yan: readonly variable
[trial@localhost cuichangjian]$ echo $yan
14
[trial@localhost cuichangjian]$ unset yan
bash: unset: yan: cannot unset: readonly variable
```

②、变量定义规则

1. 变量名称可以由字母、数字和下划线组成，但是不能以数字开头，环境变量名建议大写
2. 等号两侧不能有空格
3. 在 bash 中，变量默认类型都是字符串类型，无法直接进行数值运算（具体在后面说明）

```shell
[trial@localhost cuichangjian]$ yuehai=18
[trial@localhost cuichangjian]$ echo $yuehai
18
[trial@localhost cuichangjian]$ $yuehai=$yuehai+1
bash: 18=18+1: command not found
[trial@localhost cuichangjian]$ $yuehai+1
bash: 18+1: command not found
[trial@localhost cuichangjian]$ yuehai=yuehai+1
[trial@localhost cuichangjian]$ echo $yuehai
yuehai+1
[trial@localhost cuichangjian]$ yuehai=1+2
[trial@localhost cuichangjian]$ echo $yuehai
1+2
[trial@localhost cuichangjian]$
```

4. 若想进行数值运算，则需要用 `$[]` 或者 `$(())` 括起来（具体在后面说明）

```shell
[trial@localhost cuichangjian]$ yuehai=$[14+16]
[trial@localhost cuichangjian]$ echo $yuehai
30
[trial@localhost cuichangjian]$ yan=$((12+14))
[trial@localhost cuichangjian]$ echo $yan
26
```

5. 变量的值如果有空格，需要使用双引号或单引号括起来

```shell
[trial@localhost cuichangjian]$ yuehai=12 34
-bash: 34: command not found
[trial@localhost cuichangjian]$ yuehai="12 34"
[trial@localhost cuichangjian]$ echo $yuehai
12 34
```

3、特殊变量

①、 `$n`

Ⅰ、语法

`$n`：

1. n 为数字
2. $0 代表该脚本名称
3. $1–$9 代表第一到第九个参数，十以上的参数需要用大括号包含，如 `${10}`

II 、案例

1. 基本使用

```shell
[trial@localhost shell]$ touch 01test.sh
[trial@localhost shell]$ vim 01test.sh
[trial@localhost shell]$ cat 01test.sh
#!/bin/bash

echo '脚本名称 $0: '$0
echo '参数1 $1: '$1
echo '参数2 $2: '$2
echo '参数3 $3: '$3
[trial@localhost shell]$ ./01test.sh
脚本名称 $0: ./01test.sh
参数1 $1:
参数2 $2:
参数3 $3:
[trial@localhost shell]$ ./01test.sh  q w e
脚本名称 $0: ./01test.sh
参数1 $1: q
参数2 $2: w
参数3 $3: e
```

②、 `$#`

I 、语法

`$#`：

1. 获取所有输入参数的个数
2. 常用于循环、判断参数的个数是否、加强脚本的健壮性

II 、案例

1. 基本使用

Shell | 复制代码

```shell
[trial@localhost shell]$ vim 01test.sh
[trial@localhost shell]$ cat 01test.sh
#!/bin/bash

echo '----$n----------------'
echo '脚本名称 $0: '$0
echo '参数1 $1: '$1
echo '参数2 $2: '$2
echo '参数3 $3: '$3

echo '----$#----------------'
echo '传入参数的个数为 $#: '$#

[trial@localhost shell]$ ./01test.sh
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1:
参数2 $2:
参数3 $3:
----$#----------------
传入参数的个数为 $#: 0

[trial@localhost shell]$ ./01test.sh q
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1: q
参数2 $2:
参数3 $3:
----$#----------------
传入参数的个数为 $#: 1

[trial@localhost shell]$ ./01test.sh q wer
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1: q
参数2 $2: wer
参数3 $3:
----$#----------------
传入参数的个数为 $#: 2

[trial@localhost shell]$
```

③、 `$*`

Ⅰ、语法

`$*`：

1. 这个变量代表命令行中所有的参数
2. `$*` 把所有的参数看成一个整体

Ⅱ、案例

1. 基本使用

Shell | 复制代码

```shell
[trial@localhost shell]$ vim 01test.sh
[trial@localhost shell]$ cat 01test.sh
#!/bin/bash

echo '----$n----------------'
echo '脚本名称 $0: '$0
echo '参数1 $1: '$1
echo '参数2 $2: '$2
echo '参数3 $3: '$3

echo '----$#----------------'
echo '传入参数的个数为 $#: '$#

echo '----$*----------------'
echo '传入的所有参数为 $*: '$*

[trial@localhost shell]$ ./01test.sh
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1:
参数2 $2:
参数3 $3:
----$#----------------
传入参数的个数为 $#: 0
----$*----------------
传入的所有参数为 $*:

[trial@localhost shell]$ ./01test.sh qqq
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1: qqq
参数2 $2:
参数3 $3:
----$#----------------
传入参数的个数为 $#: 1
----$*----------------
传入的所有参数为 $*: qqq

[trial@localhost shell]$ ./01test.sh qqq qqq wwer sadas
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1: qqq
参数2 $2: qqq
参数3 $3: wwer
----$#----------------
```

```
传入参数的个数为 $#: 4
----$*----------------
传入的所有参数为 $*: qqq qqq wwer sadas

[trial@localhost shell]$
```

④、 `$@`

Ⅰ、语法

`$@` ：

1. 这个变量也代表命令行中所有的参数
2. 不过 `$@` 把每个参数区分对待

Ⅱ、案例

1. 基本使用

Shell | 复制代码

```shell
[trial@localhost shell]$ vim 01test.sh
[trial@localhost shell]$ cat 01test.sh
#!/bin/bash

echo '----$n----------------'
echo '脚本名称 $0: '$0
echo '参数1 $1: '$1
echo '参数2 $2: '$2
echo '参数3 $3: '$3

echo '----$#----------------'
echo '传入参数的个数为 $#: '$#

echo '----$*----------------'
echo '传入的所有参数为 $*: '$*

echo '----$@----------------'
echo '传入的所有参数为 $@: '$@

[trial@localhost shell]$ ./01test.sh
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1:
参数2 $2:
参数3 $3:
----$#----------------
传入参数的个数为 $#: 0
----$*----------------
传入的所有参数为 $*:
----$@----------------
传入的所有参数为 $@:

[trial@localhost shell]$ ./01test.sh 1 23 ww qweerr
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1: 1
参数2 $2: 23
参数3 $3: ww
----$#----------------
传入参数的个数为 $#: 4
----$*----------------
传入的所有参数为 $*: 1 23 ww qweerr
----$@----------------
传入的所有参数为 $@: 1 23 ww qweerr
```

⑤、 `$?`

Ⅰ、语法

`$?`：

1. 最后一次执行的命令的返回状态
2. 如果这个变量的值为 0，证明上一 个命令正确执行
3. 如果这个变量的值为非 0（具体是哪个数，由命令自己来决定），则证明 上一个命令执行不正确了

Ⅱ、案例

1. 基本使用

```shell
[trial@localhost shell]$ ./01test.sh
----$n----------------
脚本名称 $0: ./01test.sh
参数1 $1:
参数2 $2:
参数3 $3:
----$#----------------
传入参数的个数为 $#: 0
----$*----------------
传入的所有参数为 $*:
----$@----------------
传入的所有参数为 $@:

[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ ./001test.sh
-bash: ./001test.sh: No such file or directory
[trial@localhost shell]$ echo $?
127
```

四、运算符

1、语法

1. `expr`：不推荐
2. 方括号：`[运算式]`
3. 小括号：`((运算式))`

2、案例

1. 基本用法

```shell
[trial@localhost shell]$ expr 1 + 2
3
[trial@localhost shell]$ expr 1 + 2 + 4
7

[trial@localhost shell]$ s=$[ (2 + 3) *4 ]
[trial@localhost shell]$ echo $s
20

[trial@localhost shell]$ s2=$(( ((2 + 3) *4) - 1 ))
[trial@localhost shell]$ echo $s2
19
```

2. 命令替换：`a=$(运算式)` 或 `a=`运算式``；将一个命令执行的结果拿过来进行操作；不推荐

```shell
[trial@localhost shell]$ echo $s

[trial@localhost shell]$ s=2
[trial@localhost shell]$ echo $s
2
[trial@localhost shell]$ s=$(expr 1 + 2)
[trial@localhost shell]$ echo $s
3
```

3. 脚本的使用

```shell
[trial@localhost shell]$ vim 02test.sh
[trial@localhost shell]$ chmod 755 02test.sh
[trial@localhost shell]$ cat 02test.sh
#!/bin/bash

# $1: 传入的第一个参数
# 21: 传入的第二个参数
sum=$[$1 + $2]
echo sum=$sum
[trial@localhost shell]$ ./02test.sh 1 2
sum=3
```

五、条件判断

1、基本语法

①、`test condition`

`test 变量 = 值`

1. 返回 0 代表正确， true
2. 返回 1 代表错误， false

```shell
[trial@localhost shell]$ echo $s
3

[trial@localhost shell]$ test $s = 3
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ test $s = 4
[trial@localhost shell]$ echo $?
1
```

②、`[ condition ]`

`[ 变量 = 值 ]` 或 `[ 字符串 ]`

1. 注意 condition 前后要有空格
2. test condition 的简写
3. `[ 变量 = 值 ]`：
    a. 返回 0 代表正确， true
    b. 返回 1 代表错误， false

```shell
[trial@localhost shell]$ echo $s
3

[trial@localhost shell]$ [ $s = 3 ]
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ [ $s = 5 ]
[trial@localhost shell]$ echo $?
1
```

4. `[ 字符串 ]`：
    a. 为空返回 1 ， false
    b. 不为空 0 ， true

```shell
[trial@localhost shell]$ [  ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$ [ skdje415d45e4as ]
[trial@localhost shell]$ echo $?
0
```

2、常用判断条件

①、两个整数之间比较

1. `-eq` 等于（equal）

```shell
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ [ $s = 5 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$ [ $s = 3 ]
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ [ $s -eq 5 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$ [ $s -eq 3 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$
```

2. `-ne` 不等于（not equal）

```shell
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ [ $s != 3 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$ [ $s != 5 ]
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ [ $s -ne 3 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$ [ $s -ne 5 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$
```

3. `-lt` 小于（less than）

```shell
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ [ $s < 3 ]
-bash: 3: No such file or directory

[trial@localhost shell]$ [ $s -lt 3 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$ [ $s -lt 5 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$
```

4. `-le` 小于等于（less equal）
5. `-gt` 大于（greater than）
6. `-ge` 大于等于（greater equal）
7. 如果是字符串之间的比较，用等号 `=` 判断相等；用 `!=` 判断不等
8. 与或非
    a. 与：`[ 条件1 -a 条件2 ]`，全对才为对；表示前一条命令执行成功时，才执行后一条命令

```shell
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ [ $s -ge 3 -a $s -le 5 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ $s -ge 3 -a $s -eq 5 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$
```

    b. 或：`[ 条件1 -o 条件2 ]`，全错才为错；表示上一条命令执行失败后，才执行下一条命令

```shell
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ [ $s -ge 3 -o $s -eq 5 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ $s -eq 3 -o $s -eq 5 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ $s != 3 -o $s == 5 ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$
```

c. 非：`[ !条件 ]`，对变错，错变对；使用括号需要用转义符转移

```shell
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ [ ! $s != 3 ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ ! \($s != 3 -o $s == 5\) ]
[trial@localhost shell]$ echo $?
1
[trial@localhost shell]$
```

②、按照文件权限进行判断

1. `-r` 有读的权限（read）
2. `-w` 有写的权限（write）
3. `-x` 有执行的权限（execute）

```shell
[trial@localhost shell]$ ll
total 12
-rwxr-xr-x 1 trial trial  33 Aug 10 16:50 00test.sh
-rwxr-xr-x 1 trial trial 353 Aug 12 12:04 01test.sh
-rwxr-xr-x 1 trial trial 106 Aug 15 16:03 02test.sh
[trial@localhost shell]$ [ -r 00test.sh ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ -w 00test.sh ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ -x 00test.sh ]
[trial@localhost shell]$ echo $?
0
```

③、按照文件类型进行判断

1. `-e` 文件存在（existence）
2. `-f` 文件存在并且是一个常规的文件（file）
3. `-d` 文件存在并且是一个目录（directory）

```shell
[trial@localhost shell]$ ll
total 12
-rwxr-xr-x 1 trial trial  33 Aug 10 16:50 00test.sh
-rwxr-xr-x 1 trial trial 353 Aug 12 12:04 01test.sh
-rwxr-xr-x 1 trial trial 106 Aug 15 16:03 02test.sh
[trial@localhost shell]$ [ -e 00test.sh ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ -f 00test.sh ]
[trial@localhost shell]$ echo $?
0
[trial@localhost shell]$ [ -d 00test.sh ]
[trial@localhost shell]$ echo $?
1
```

④、与 `&&` 或 `||` 操作

Ⅰ、逻辑与：`&&`

1. 逻辑与 `&&` ：全对才为对；表示前一条命令执行成功时，才执行后一条命令

```shell
[trial@localhost shell]$ [ $s = 3 ] && [ $s -lt 5 ] && [ -e 00test.sh ]
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ [ $s != 3 ] && [ $s -lt 5 ] && [ -e 00test.sh ]
[trial@localhost shell]$ echo $?
1
```

2. 在 shell 中使用时，只有 `&&` 左边的命令执行为真，右边的命令才会被执行；0 为真，1 为假

```shell
[trial@localhost shell]$ cat shell-or-test.sh
#!/bin/bash

# 只有在 && 左边的命令返回真（命令返回值 $? == 0），&& 右边的命令才会被执行
test1(){
        return 0
}
test2(){
        return 1
}
test1 && echo "test1 && Hello World !"
test2 && echo "test2 && Hello World !"
[trial@localhost shell]$ ./shell-or-test.sh
test1 && Hello World !
[trial@localhost shell]$
```

Ⅱ、逻辑或：`||`

1. 逻辑或 `||` ：全错才为错；表示上一条命令执行失败后，才执行下一条命令

```shell
[trial@localhost shell]$ [ $s = 3 ] || [ $s -lt 5 ] || [ -e 00test.sh ]
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ [ $s != 3 ] || [ $s -lt 5 ] || [ -e 00test.sh ]
[trial@localhost shell]$ echo $?
0

[trial@localhost shell]$ [ $s != 3 ] || [ $s -lt 3 ] || [ -d 00test.sh ]
[trial@localhost shell]$ echo $?
1
```

2. 在 shell 中使用时，只有 `||` 左边的命令执行为假，右边的命令才会被执行；0 为真，1 为假

```shell
[trial@localhost shell]$ cat shell-and-test
#!/bin/bash

# 只有在 || 左边的命令返回假（命令返回值 $? == 1），&& 右边的命令才会被执行
test3(){
        return 0
}
test4(){
        return 1
}
test3 || echo "test3 && Hello World !"
test4 || echo "test4 && Hello World !"

[trial@localhost shell]$ ./shell-and-test
test4 && Hello World !
[trial@localhost shell]$
```

III、三元运算符

1. 三元运算符

```shell
[trial@localhost shell]$ [ $s = 3 ] && echo "正确" || echo "错误"
正确
[trial@localhost shell]$ [ $s != 3 ] && echo "正确" || echo "错误"
错误
```

六、流程控制

1、if 判断

①、语法

Ⅰ、单分支

```shell
if [ 条件判断式 ];then
  程序
fi

# 或

if [ 条件判断式 ]
then
  程序
fi
```

Ⅱ、多分支

```shell
if [ 条件判断式 ];then
  程序
elif [ 条件判断式 ];then
  程序
else
  程序
fi
```

②、注意事项

1. `[ 条件判断式 ]`，中括号和条件判断式之间必须有空格
2. `if` 后要有空格

③、案例

1. 判断变量 `s=3`

```shell
[trial@localhost shell]$ s=3
[trial@localhost shell]$ echo $s
3
[trial@localhost shell]$ if [ $s = 3 ];then echo s=3; fi
s=3
[trial@localhost shell]$
```

2. 判断传入的参数；为空时会报错

```shell
[trial@localhost shell]$ vim 03test.sh
[trial@localhost shell]$ cat 03test.sh
#!/bin/bash

if [ $1 = yuehai ];then
        echo "月海"
fi
[trial@localhost shell]$ chmod 755 03test.sh
[trial@localhost shell]$ ./03test.sh
./03test.sh: line 3: [: =: unary operator expected
[trial@localhost shell]$ ./03test.sh yuehai
月海
[trial@localhost shell]$
```

3. 判断传入的参数；将参数用 `""` 括起来，并在后面加上 `x` ，这样传参为空时就为： `x=yuehai x` ，不会报错

```shell
[trial@localhost shell]$ vim 03test.sh
[trial@localhost shell]$ cat 03test.sh
#!/bin/bash

if [ "$1"x = "yuehai"x ];then
        echo "月海"
fi
[trial@localhost shell]$ chmod 755 03test.sh
[trial@localhost shell]$ ./03test.sh
./03test.sh: line 3: [: =: unary operator expected
[trial@localhost shell]$ ./03test.sh yuehai
月海
[trial@localhost shell]$
```

4. `else`

```shell
[trial@localhost shell]$ cat 03test.sh
#!/bin/bash

if [ "$1"x = "yuehai"x ];then
        echo "月海"
else
        echo "不知道输入的啥"
fi
[trial@localhost shell]$ ./03test.sh yuehai
月海
[trial@localhost shell]$ ./03test.sh
不知道输入的啥
[trial@localhost shell]$ ./03test.sh sdjf
不知道输入的啥
[trial@localhost shell]$
```

5. 多分支

```shell
[trial@localhost shell]$ cat 03test.sh
#!/bin/bash

if [ "$1"x = "yuehai"x ];then
        echo "月海"
elif [ "$1"x = "yan"x ];then
        echo "言"
elif [ "$1"x = "yu"x ];then
        echo "羽"
else
        echo "不知道输入的啥"
fi
[trial@localhost shell]$ ./03test.sh yuehai
月海
[trial@localhost shell]$ ./03test.sh yan
言
[trial@localhost shell]$ ./03test.sh yu
羽
[trial@localhost shell]$ ./03test.sh
不知道输入的啥
[trial@localhost shell]$ ./03test.sh fsdfwq
不知道输入的啥
[trial@localhost shell]$
```

2、case 语句

①、语法

```shell
case $变量名 in
"值 1")
  如果变量的值等于值 1，则执行程序 1
;;
"值 2")
  如果变量的值等于值 2，则执行程序 2
;;
  …省略其他分支…
*)
  如果变量的值都不是以上的值，则执行此程序
;;
esac
```

②、注意事项

1. case 行尾必须为单词 `in`，每一个模式匹配必须以右括号 `)` 结束
2. 双分号 `;;` 表示命令序列结束，相当于 java 中的 break
3. 最后的 `*)` 表示默认模式，相当于 java 中的 default

③、案例

1. 基本用法

```shell
[trial@localhost shell]$ cat 04test.sh
#!/bin/bash

case $1 in
1)
        echo "1"
;;
2)
        echo "2"
;;
yuehai)
        echo "月海"
;;
yan)
        echo "言"
;;
*)
        echo "也不知道说的啥"
;;
esac
[trial@localhost shell]$ ./04test.sh 1
1
[trial@localhost shell]$ ./04test.sh 2
2
[trial@localhost shell]$ ./04test.sh yuehai
月海
[trial@localhost shell]$ ./04test.sh yan
言
[trial@localhost shell]$ ./04test.sh
也不知道说的啥
[trial@localhost shell]$ ./04test.sh fljqa
也不知道说的啥
[trial@localhost shell]$
```

3、while 循环

①、语法

```shell
while [ 条件判断式 ]
do
  程序
done
```

②、注意事项

1. `[ 条件判断式 ]`，中括号和条件判断式之间必须有空格
2. `while` 后要有空格

③、案例

1. 一般用法

```shell
[trial@smartedu shell]$ cat 07test.sh
#!/bin/bash

# 1-100 的和
sum=0
i=0
while [ $i -le 100 ]
do
        sum=$[ $sum + $i ]
        i=$[ $i + 1 ]
done
echo 1-100 的和是: $sum
[trial@smartedu shell]$ ./07test.sh
1-100 的和是: 5050
[trial@smartedu shell]$
```

4、for 循环

1. shell 中 `{}` 是序列的意思，即区间；`{1..100}` 的意思是：1-100 这 100 个数
2. 运算符 `[ ]` 中不能有 `>、<、!` 等符号，只能是 `-gt、-lt、-ne` 等
3. 运算符 `(( ))` 可以有 `>、<、!` 等符号

①、语法

1. 条件循环

```shell
for (( 初始值;循环控制条件;变量变化 ))
do
  程序
done
```

2. 循环值

```shell
for 变量 in 值 1 值 2 值 3…
do
  程序
done
```

②、注意事项：`$*` 和 `$@` 的区别

1. `$*` 和 `$@` 都表示传递给函数或脚本的所有参数
2. 不被双引号 `""` 包含时，都以 `$1` 、 `$2` 、 `…` 、 `$n` 的形式输出所有参数

```shell
[atguigu@hadoop101 shells]$ vim for3.sh
#!/bin/bash

echo '=============$*=============='

for i in $*
do
  echo "ban zhang love $i"
done

echo '=============$@=============='

for j in $@
do
  echo "ban zhang love $j"
done

[atguigu@hadoop101 shells]$ chmod 777 for3.sh
[atguigu@hadoop101 shells]$ ./for3.sh cls mly wls
=============$*=============
banzhang love cls
banzhang love mly
banzhang love wls
=============$@=============
banzhang love cls
banzhang love mly
banzhang love wls
```

3. 当它们被双引号 `""` 包含时，
   a. `$*` 会将所有的参数作为一个整体，以 `$1 $2 …$n` 的形式输出所有参数
   b. `$@` 会将各个参数分开，以 `$1` 、 `$2` 、 `…` 、 `$n` "的形式输出所有参数

Shell 复制代码

```shell
[atguigu@hadoop101 shells]$ vim for4.sh
#!/bin/bash

echo '=============$*=============='

for i in "$*"
# $* 中的所有参数看成是一个整体，所以这个 for 循环只会循环一次
do
  echo "ban zhang love $i"
done

echo '=============$@=============='

for j in "$@"
# $@ 中的每个参数都看成是独立的，所以“$@”中有几个参数，就会循环几次
do
  echo "ban zhang love $j"
done

[atguigu@hadoop101 shells]$ ./for4.sh cls mly wls
=============$*==============
banzhang love cls mly wls
=============$@==============
banzhang love cls
banzhang love mly
banzhang love wls
```

③、案例

1. 条件循环

```shell
[trial@smartedu shell]$ cat 05.test.sh
#!/bin/bash

# 1-100 的和
sum=0
for (( i=0;i<=100;i++ ))
do
        sum=$[ $sum+$i ]
done

echo "1-100 的和为: "$sum

# 传入的两个参数之间说有数的和
sum2=0
for (( i=$1;i<=$2;i++ ))
do
        sum2=$[ $sum2+$i ]
done

echo "传入的两个参数之间的所有的和为: "$sum2

[trial@smartedu shell]$ ./05.test.sh
1-100 的和为: 5050
./05.test.sh: line 14: ((: i=: syntax error: operand expected (error toke
n is "=")
传入的两个参数之间的所有的和为: 0

[trial@smartedu shell]$ ./05.test.sh 1 100
1-100 的和为: 5050
传入的两个参数之间的所有的和为: 5050

[trial@smartedu shell]$
```

2. 循环值

Shell | 复制代码

```shell
[trial@smartedu shell]$ cat 06test.sh
#!/bin/bash

# 打印固定值
num=0
for i in 1 2 yuehai 121 yan_yu
do
        num=$[ $num + 1 ]
        echo "第 $num 次循环的值是: "$i
done

echo ----------------------------------------

# 打印传入的参数
num=0
for i in $1 $2 $3 $4
do
        num=$[ $num + 1 ]
        echo "第 $num 次循环的值是: "$i
done
[trial@smartedu shell]$ ./06test.sh
第 1 次循环的值是: 1
第 2 次循环的值是: 2
第 3 次循环的值是: yuehai
第 4 次循环的值是: 121
第 5 次循环的值是: yan_yu
----------------------------------------
[trial@smartedu shell]$ ./06test.sh 1 2 3 4
第 1 次循环的值是: 1
第 2 次循环的值是: 2
第 3 次循环的值是: yuehai
第 4 次循环的值是: 121
第 5 次循环的值是: yan_yu
----------------------------------------
第 1 次循环的值是: 1
第 2 次循环的值是: 2
第 3 次循环的值是: 3
第 4 次循环的值是: 4
[trial@smartedu shell]$
```

七、read 读取控制台输入

1、语法

`read（参数）（变量名）`

①、参数

1. –p：指定读取值时的提示符
2. –t：指定读取值时等待的时间（秒）如果 –t 不加表示一直等待

②、变量名

指定读取值的变量名

2、案例

1. 一般使用

Shell | 复制代码

```shell
[trial@smartedu shell]$ cat 08test.sh
#!/bin/bash

read -t 10 -p "请输入：" name

echo 您输入的字符为：$name
[trial@smartedu shell]$ ./08test.sh
请输入：swe123
您输入的字符为：swe123
[trial@smartedu shell]$
```

八、函数

1、系统函数

比较常见的两个

①、 basename

Ⅰ、语法

1. `basename [string / pathname] [suffix]`
2. basename 命令会删掉所有的前缀，包括最后一个（‘/’）字符，然后将字符串显示出来。
3. basename 可以理解为取路径里的文件名称

Ⅱ、参数

1. suffix 为后缀，如果 suffix 被指定了，basename 会将 string 或 pathname 中的 suffix 去掉。

Ⅲ、案例

1. 去掉前缀

Shell | 复制代码

```shell
[trial@smartedu shell]$ ll
total 48
-rwxr-xr-x. 1 trial trial   33  8月 10 16:50 00test.sh
-rwxr-xr-x. 1 trial trial  353  8月 12 12:04 01test.sh
-rwxr-xr-x. 1 trial trial  106  8月 15 16:03 02test.sh
-rwxr-xr-x. 1 trial trial  191  8月 18 17:57 03test.sh
-rwxr-xr-x. 1 trial trial  143  8月 19 10:09 04test.sh
-rwxr-xr-x. 1 trial trial  291  8月 23 09:44 05test.sh
-rwxr-xr-x. 1 trial trial  500  8月 23 13:50 06test.sh
-rwxr-xr-x. 1 trial trial  130  8月 23 15:46 07test.sh
-rwxr-xr-x. 1 trial trial   82  8月 23 17:10 08test.sh
-rw-r--r--. 1 trial trial 1093  8月 18 16:10 50test.sh
-rwxr-xr-x. 1 trial trial  252  8月 23 14:26 shell-and-test.sh
-rwxr-xr-x. 1 trial trial  245  8月 23 14:27 shell-or-test.sh
[trial@smartedu shell]$ pwd
/home/trial/SYS/cuichangjian/shell
[trial@smartedu shell]$ basename /home/trial/SYS/cuichangjian/shell/00test.sh
00test.sh
[trial@smartedu shell]$
```

2. 去掉前缀和指定的后缀

```shell
[trial@smartedu shell]$ ll
total 48
-rwxr-xr-x. 1 trial trial   33  8月 10 16:50 00test.sh
-rwxr-xr-x. 1 trial trial  353  8月 12 12:04 01test.sh
-rwxr-xr-x. 1 trial trial  106  8月 15 16:03 02test.sh
-rwxr-xr-x. 1 trial trial  191  8月 18 17:57 03test.sh
-rwxr-xr-x. 1 trial trial  143  8月 19 10:09 04test.sh
-rwxr-xr-x. 1 trial trial  291  8月 23 09:44 05test.sh
-rwxr-xr-x. 1 trial trial  500  8月 23 13:50 06test.sh
-rwxr-xr-x. 1 trial trial  130  8月 23 15:46 07test.sh
-rwxr-xr-x. 1 trial trial   82  8月 23 17:10 08test.sh
-rw-r--r--. 1 trial trial 1093  8月 18 16:10 50test.sh
-rwxr-xr-x. 1 trial trial  252  8月 23 14:26 shell-and-test.sh
-rwxr-xr-x. 1 trial trial  245  8月 23 14:27 shell-or-test.sh
[trial@smartedu shell]$ pwd
/home/trial/SYS/cuichangjian/shell
[trial@smartedu shell]$ basename /home/trial/SYS/cuichangjian/shell/00test.sh .sh
00test
[trial@smartedu shell]$
```

②、 dirname

Ⅰ、语法

1. `dirname 文件绝对路径`
2. 从给定的包含绝对路径的文件名中去除文件名 （非目录的部分），然后返回剩下的路径（目录的部分）
3. dirname 可以理解为取文件路径的绝对路径名称

Ⅱ、参数

无

Ⅲ、案例

```shell
[trial@smartedu shell]$ ll
total 48
-rwxr-xr-x. 1 trial trial   33  8月 10 16:50 00test.sh
-rwxr-xr-x. 1 trial trial  353  8月 12 12:04 01test.sh
-rwxr-xr-x. 1 trial trial  106  8月 15 16:03 02test.sh
-rwxr-xr-x. 1 trial trial  191  8月 18 17:57 03test.sh
-rwxr-xr-x. 1 trial trial  143  8月 19 10:09 04test.sh
-rwxr-xr-x. 1 trial trial  291  8月 23 09:44 05test.sh
-rwxr-xr-x. 1 trial trial  500  8月 23 13:50 06test.sh
-rwxr-xr-x. 1 trial trial  130  8月 23 15:46 07test.sh
-rwxr-xr-x. 1 trial trial   82  8月 23 17:10 08test.sh
-rw-r--r--. 1 trial trial 1093  8月 18 16:10 50test.sh
-rwxr-xr-x. 1 trial trial  252  8月 23 14:26 shell-and-test.sh
-rwxr-xr-x. 1 trial trial  245  8月 23 14:27 shell-or-test.sh
[trial@smartedu shell]$ pwd
/home/trial/SYS/cuichangjian/shell
[trial@smartedu shell]$ dirname /home/trial/SYS/cuichangjian/shell/08test.sh
/home/trial/SYS/cuichangjian/shell
[trial@smartedu shell]$
```

2、自定义函数

①、 语法

```shell
[ function ] funname[()]
{
    Action;
    [return int;]
}
```

```shell
[ function ] 函数名称[()]
{
    函数体;
    [return int;]
}
```

②、 经验技巧

1. 必须在调用函数地方之前，先声明函数，shell 脚本是逐行运行。不会像其它语言一样先编译。
2. 函数返回值，只能通过 `$?` 系统变量获得
3. 若有 `return` 则通过 `$?` 可获得其设置的返回值； `return` 后跟数值 n(0-255)
4. 如果不加 `return` ，将以最后一条命令运行结果，作为返回值

③、案例

1. 获取控制台输入，将两数相加

Shell | 复制代码

```shell
#!/bin/bash

# 获取用户输入的参数
read -t 10 -p "请输入第一个加数: " a
read -t 10 -p "请输入第二个加数: " b

# 编写函数，传递两个参数，使其相加
function add () {
    sum=$(( $1 + $2 ))
    echo 两个数的和为: $sum
    return 0
}

# 调用函数
add "$a" "$b"
```

Shell | 复制代码

```shell
[trial@smartedu shell]$ ./09test.sh
请输入第一个加数: 33
请输入第二个加数: 1
两个数的和为: 34
[trial@smartedu shell]$ ./09test.sh
请输入第一个加数: 5
请输入第二个加数: 21
两个数的和为: 26
[trial@smartedu shell]$
```

九、文本处理工具

1、cut

①、语法

`cut [选项参数] filename`

②、参数

1. –f：列号，提取第几列
2. –d：分隔符，按照指定分隔符分割列，默认是制表符“\t”
3. –c：按字符进行切割 后加加 n 表示取第几列 比如 –c 1

③、说明

cut 的工作就是“剪”，具体的说就是在文件中负责剪切数据用的。cut 命令从文件的每一行剪切字节、字符和字段并将这些字节、字符和字段输出。

④、案例

1. 提取出第一列，指定以空格 `  ` 分隔列

```shell
[trial@smartedu data]$ cat data
3 2 1
2 2 2
1 2 3
1 3 3
1 1 1
2 2 2
3 3 3
2 4 6
1 4 6
3 2 1
2 1 4
3 3 3
2 1 3
[trial@smartedu data]$ cut -d " " -f 1 data
3
2
1
1
1
2
3
2
1
3
2
3
2
[trial@smartedu data]$
```

2. ·提取出第一列至第二列，指定以空格 `  ` 分隔列

```shell
[trial@smartedu data]$ cat data
3 2 1
2 2 2
1 2 3
1 3 3
1 1 1
2 2 2
3 3 3
2 4 6
1 4 6
3 2 1
2 1 4
3 3 3
2 1 3
[trial@smartedu data]$ cut -d " " -f 1,2 data
3 2
2 2
1 2
1 3
1 1
2 2
3 3
2 4
1 4
3 2
2 1
3 3
2 1
[trial@smartedu data]$
```

3. 选取系统 PATH 变量值，第 2 个“：”开始后的所有路径

```shell
[trial@smartedu data]$ echo $PATH
/home/SMART_TRIAL:/home/SMART:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
[trial@smartedu data]$ echo $PATH | cut -d ":" -f3-
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
[trial@smartedu data]$
```

2、awk

https://www.yuque.com/yuehai-igspb/xuexi/ov59i7#qCrO0

十、综合应用案例

1、归档（备份）文件

1. 实际生产应用中，往往需要对重要数据进行归档备份。
2. 需求：实现一个每天对指定目录归档备份的脚本，输入一个目录名称（末尾不带/），将目录下所有文件按天归档保存，并将归档日期附加在归档文件名上，放在 `/home/trial/SYS/cuichangjian/shell/case/case/01case` 下。
3. 这里用到了归档命令： `tar`
4. 后面可以加上 `-c` 选项表示归档，加上 `-z` 选项表示同时进行压缩，得到的文件后缀名为 `.tar.gz`

---

1. 归档脚本

Shell | 复制代码

```shell
#!/bin/bash

# 判断输入参数个数是否为 1
if [ $# -ne 1 ]; then
    echo "参数个数错误！应该输入一个参数，作为归档目录名"
    # 退出脚本
    exit
fi

# 判断文件存在并且是一个目录
if [ -d "$1" ]; then
    # 文件存在并且是一个目录
    echo "文件存在并且是一个目录"
else
    # 文件不存在或不是一个目录
    echo "文件不存在或不是一个目录"
    # 退出脚本
    exit
fi

# 从参数中获取当前目录名
DIR_NAME=$( basename "$1" )
# 从参数中获取路径，然后使用 pwd 命令获取全路径名
DIR_PATH=$( cd $(dirname "$1") ; pwd )

# 获取当前日期
DATE=$( date +"%Y%m%d" )

# 定义生成的归档文件名称
FILE=archive_${DIR_NAME}_${DATE}.tar.gz
# 定义生成的归档文件名称和文件路径
DEST=/home/trial/SYS/cuichangjian/shell/case/01case/${FILE}

# 开始归档
echo "开始归档..."
tar -czf "$DEST" "$DIR_PATH"/"$DIR_NAME"

# 判断归档是否成功
if [ $? -eq 0 ]; then
    echo "归档成功，文件为：$DEST"
else
    echo "归档失败"
    exit
fi

```

```
exit
```

```shell
[trial@smartedu case]$ ./01case.sh ./
文件存在并且是一个目录
开始归档...
tar: Removing leading `/' from member names
归档成功，文件为：/home/trial/SYS/cuichangjian/shell/case/01case/archive_._20220824.tar.gz
[trial@smartedu case]$
```

2. 定时执行脚本
   a. 使用 `crontab -e` 命令执行文字编辑器来设定时程表
   b. `crontab` 语法说明

```shell
# 分 时 天 月 周几 执行的脚本 传递的参数
# 每天的 2:00 执行 脚本：01case.sh，并给脚本传递参数：../case
0 2 * * * 01case.sh ../case
```

   c. –l : 列出目前的时程表
   d. –r : 删除目前的时程表

2、发送消息

1. 我们可以利用 Linux 自带的 mesg 和 write 工具，向其它用户发送消息。
2. 需求：实现一个向某个用户快速发送消息的脚本，输入用户名作为第一个参数，后面直接跟要发送的消息。脚本需要检测用户是否登录在系统中、是否打开消息功能，以及当前发

送消息是否为空。